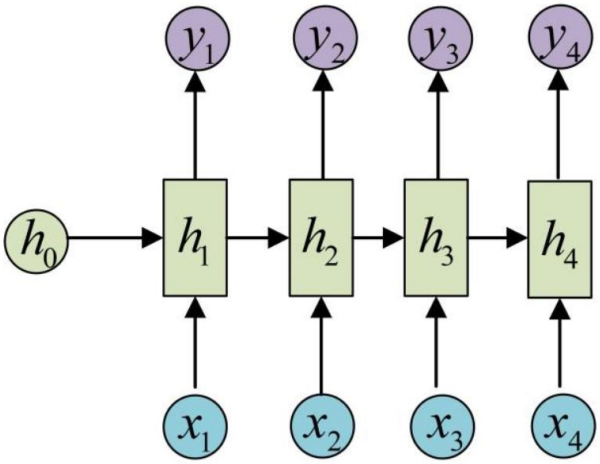
2. RNN、LSTM 和 GRU

RNN：

循环神经网络 RNN 是一种适合处理序列数据的神经网络，主要特点是引入隐状态 hidden_state，对序列形的数据提取特征，接着再转换为输出，体现出它的记忆功能和处理时序信息的优势。除此以外 rnn 各层的参数都是相同的，也是显著特点之一。



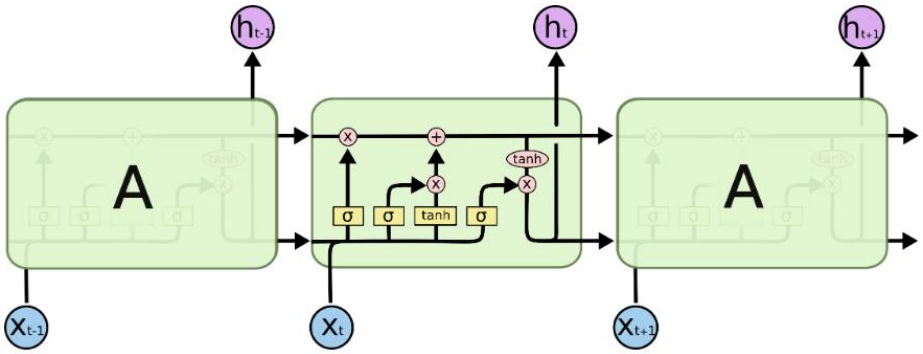
$$h_t = f(Ux_t + Wh_{t-1} + b)$$

$$y_t = Softmax(Vh_t + c)$$

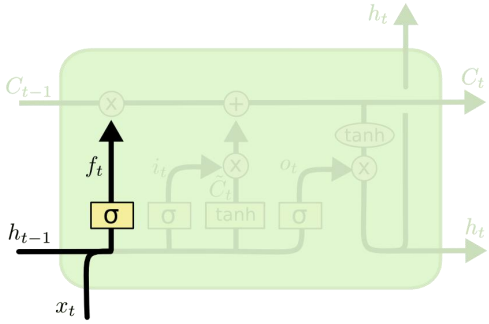
当输入序列较长时，RNN 会出现梯度消失或梯度爆炸的问题，使得它很难有效学习长距离依赖关系，信息较远的话很可能减弱或消失。

LSTM

LSTM 即长短时记忆网络，是一种 RNN 特殊的类型，可以学习长期依赖信息，在普通 RNN 的基础上引入了门控机制，包括遗忘门、输入门和输出门。遗忘门决定上一时刻记忆中哪些信息需要保留，输入门决定当前输入中的哪些信息需要写入记忆单元，输出门决定当前时刻向外输出哪些信息。



遗忘门：通过 Sigmoid 激活函数映射到 0-1，决定遗忘程度。

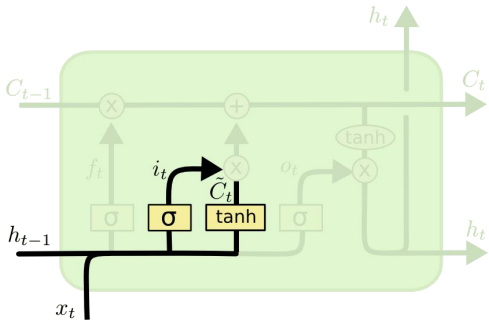


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \ + \ b_f\right)$$

输入门：确定什么样的新信息被存放在细胞状态中

sigmoid 层称输入门层，决定什么值将要更新

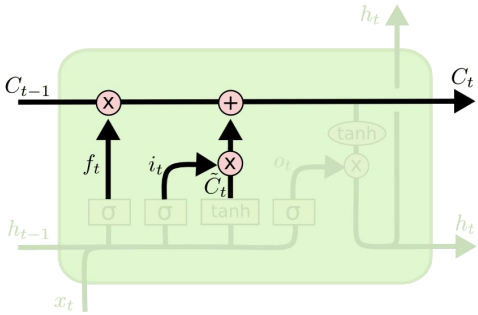
tanh 层创建一个新的候选值向量$\tilde{C}_t$，会被加入到状态中



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

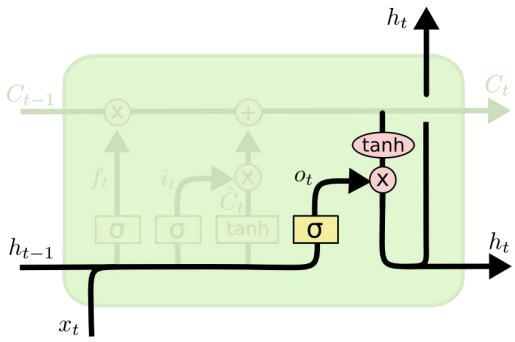
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

LSTM 关键在于细胞状态，直接在整个链上运行，只有一些少量的线性交互。信息在上面流传保持不变会很容易。



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

输出门：运行一个 sigmoid 层来确定细胞状态的哪个部分将输出出去，把细胞状态通过 tanh 进行处理（得到一个在-1 到 1 之间的值）并将它和 sigmoid 门的输出相乘，最终仅仅会我们确定输出的那部分



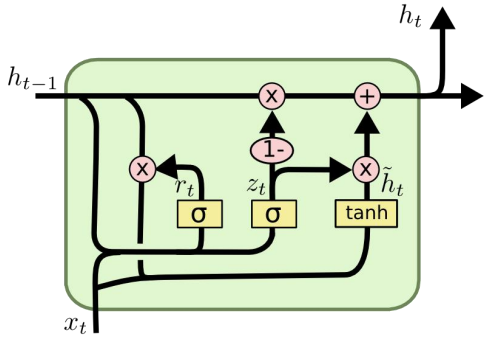
$$o_t = \sigma\left(W_o \ [h_{t-1}, x_t] \ + \ b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

通过门控机制和长期（cell_state）短期（hidden_state）记忆并行，有效缓解了 rnn 中梯度消失或梯度爆炸的问题，增强了记忆能力。

GRU

GRU 是在 LSTM 基础上进一步简化得到的一种门控循环网络，将 LSTM 中较复杂的门控结构做了合并，只保留了更新门和重置门，参数更少、结构更简单、训练速度更快。



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

3. 诗歌生成过程

3.1 数据准备与预处理

先从诗歌文件中读取每一首诗，然后对内容做预处理，比如去掉空格、过滤掉不合适的文本，把太短或者太长的内容去掉，只保留比较适合训练的诗句并在每首诗加上一个开始标记和结束标记。这样模型在训练的时候，就能知道这首诗的起点和终点。

接着，把诗里的每个字都变成对应的编号。先把所有出现过的字统计出来，建立一个字到编号的映射字典。这样中文诗就可以表示成一个整数序列。处理完成以后，所有诗歌数据就会被整理成一个二维数组，每一行代表一首诗，每个位置存放的是对应汉字的编号。

3.2 构建诗歌生成模型

模型的任务就是根据前面已经出现的字，去预测后面最有可能出现哪个字。

模型的第一部分是词嵌入层。将编号转换成向量。在词嵌入之后，数据会进入循环神经网络层去，这里是 LSTM。LSTM 得到输出以后，接入一个全连接层，把隐藏状态映射到整个字表空间中。最后模型会输出一个概率分布，表示下一个字最有可能是什么。损失函数选用通过负对数似然损失函数，反向传播调整参数。

3.3 模型训练

训练时，将训练数据划分为多个 batch，每个 batch 包含一组诗歌序列。计算模型的损失并进行梯度更新，降低 loss，提高模型生成准确性。

3.4 诗歌生成

给模型一个起始字，比如“日”这样的字，作为整首诗的开头。模型先根据这个开头预测下一个字，然后再把新生成的字接到后面，继续作为新的输入，再预测下一个字。这个过程会一直持续下去，直到模型生成了结束标记，或者整首诗的长度已经达到设定的上限，生成才会停止。最后，把得到的字符序列整理一下输出出来，就形成了一首自动生成的诗句。

4. 生成结果

Pytorch:

训练过程

```
, 3]
b_y       [333, 23, 382, 598, 436, 6, 25, 0, 62, 43, 154, 33, 36, 380, 131, 1, 183, 17, 143, 20, 532, 45, 272, 0, 916, 75, 223,
 8, 177, 185, 105, 1, 54, 27, 1430, 2222, 15, 180, 48, 0, 347, 322, 427, 162, 4, 521, 1025, 1, 101, 704, 10, 59, 66, 29, 252, 0,
 106, 22, 7, 53, 120, 1078, 321, 1, 3, 3]
******************************
epoch   5 batch number 301 loss is:  5.8026838302612305
prediction [10, 42, 9, 219, 4, 15, 47, 0, 10, 159, 9, 5, 4, 110, 22, 1, 10, 130, 9, 23, 9, 10, 64, 0, 4, 468, 92, 70, 49, 123, 1
50, 1, 10, 88, 82, 161, 24, 22, 16, 0, 53, 145, 7, 115, 10, 10, 178, 1, 128, 49, 4, 14, 9, 9, 39, 0, 4, 708, 9, 451, 92, 9, 20,
1, 3, 3]
b_y       [120, 131, 121, 64, 99, 15, 37, 0, 292, 48, 1513, 518, 460, 18, 27, 1, 963, 149, 227, 23, 9, 705, 192, 0, 144, 468, 1
0, 8, 4, 156, 150, 1, 7, 925, 274, 252, 979, 332, 85, 0, 53, 3575, 200, 554, 685, 166, 178, 1, 31, 142, 129, 35, 799, 47, 39, 0,
 101, 253, 581, 451, 49, 56, 20, 1, 3, 3]
******************************
epoch   5 batch number 302 loss is:  5.7265520095825195
prediction [10, 35, 15, 6, 10, 31, 476, 0, 4, 58, 15, 104, 4, 26, 164, 1, 10, 9, 6, 8, 9, 150, 64, 0, 4, 49, 5, 11, 4, 39, 121,
1, 10, 45, 4, 343, 9, 36, 351, 0, 7, 23, 7, 53, 10, 103, 58, 1, 10, 14, 4, 49, 9, 32, 64, 0, 4, 114, 9, 50, 10, 72, 178, 1, 3, 3
]
b_y       [39, 29, 60, 11, 128, 92, 411, 0, 91, 5, 280, 38, 475, 5, 27, 1, 790, 180, 87, 173, 1626, 531, 8, 0, 255, 49, 176, 50
, 569, 508, 20, 1, 157, 883, 75, 343, 301, 245, 364, 0, 274, 13, 897, 53, 112, 174, 246, 1, 163, 68, 4, 51, 113, 30, 29, 0, 257,
 460, 199, 1324, 385, 10, 178, 1, 3, 3]
******************************
epoch   5 batch number 303 loss is:  5.7810540199279785
```

结果（这里把一些调试的输出注释了，比如 initial linear weight）

```
(dl) C:\Users\lenovo\Desktop\作业\chap6_RNN\tangshi_for_pytorch>python main.py
风光不可辨，风月夜无声。
云树不成秋，风花月下听。
####################################
花飞不可见，山在玉门关。
水色生新月，风光满树香。
####################################
一声清风起，秋风起清风。
####################################
寒光落落日，轻竹夜光声。
白日生红树，红尘动翠微。
####################################
夜色生风在，光风起翠微。
云光初白雪，风月下青松。
####################################
一枝红树千年雪，一夜无声一夜声。
####################################
月中夜寒窗外月，寒光生不生。
云开寒色满，风动夜声明。
####################################
```

Tensorflow

生成过程

```python
def gen_sentence(begin_word):
    if begin_word not in word2id:
        raise ValueError('begin_word not in vocabulary')

    state = [tf.zeros(shape=(1, 128), dtype=tf.float32)]
    cur_token = tf.constant([word2id[start_token]], dtype=tf.int32)
    _, state = model.get_next_token(cur_token, state)

    collect = [begin_word]
    cur_token = tf.constant([word2id[begin_word]], dtype=tf.int32)

    for _ in range(50):
        next_token, state = model.get_next_token(cur_token, state)
        token = id2word[int(next_token.numpy()[0])]
        if token == end_token:
            break
        if token not in {'PAD', 'UNK', start_token}:
            collect.append(token)
        cur_token = next_token

    return ''.join(collect)

begin_words = ['日', '红', '山', '夜', '湖', '海', '月']
for w in begin_words:
    print(w + ': ' + gen_sentence(w))
```

[6] Python

```
日: 日日无人事，无人不可知。
红: 红叶不知此，何人不可知。
山: 山上春风雨，春风满树开。
夜: 夜来无处处，不见此人间。
湖: 湖上春风雨，春风满树开。
海: 海上春风满，风风落日深。
月: 月落花声满，风风落日深。
```

总结：

实验结果表明，该模型已经能够完成基本的诗歌生成功能，并生成具有一定古诗风格的句子，说明模型学习到了诗歌中的部分字词组合规律和句式特征。但同时，生成结果中也存在重复现象明显、语义连贯性不足、表达较模板化等问题。由此可以看出，当前模型能够较好模仿诗歌的表面形式，但在内容质量和整体意境方面仍有不足。